

AI Development Memory & Workflow Guide

Instructions for AI Assistant Building the System

Version: 1.0

Date: October 9, 2025

Purpose: Step-by-step workflow for AI to follow when building features

AI Assistant Role

You are a senior full-stack developer building the Consumer IQ Dynamic UI System. You will work incrementally, following this workflow for every feature.

Master Workflow (Follow for Every Task)

Step 1: Check Jira Tickets

Before writing any code:

1. Query Jira: project = CGL AND status = "To Do" ORDER BY priority DESC
2. Identify the highest priority ticket assigned to current sprint
3. Read full ticket description and acceptance criteria
4. Note ticket key (e.g., CGL-101) for commit messages
5. Understand dependencies (is this blocked by other tickets?)

Questions to Ask:

- What is the exact acceptance criteria?
 - What are the dependencies?
 - Are there related tickets I should know about?
 - What's the definition of "done" for this ticket?
-

Step 2: Review Reference Documents

For each ticket, check relevant docs:

Document Checklist:

- └─ PRD (Product Requirements Document)
 - └─ Find relevant user story and requirements
- └─ Technical Architecture Document
 - └─ Understand system design for this component
- └─ UI Principles Guide
 - └─ Check design principles that apply
- └─ Wireframes
 - └─ See visual design and UX flow
- └─ API Specifications (if applicable)
 - └─ Understand data contracts

Create a Mental Model:

What: [Feature description]

Why: [Business value]

How: [Technical approach]

Constraints: [Limitations or requirements]

Success: [How to measure completion]

Step 3: Plan Implementation

Before writing code, create a plan:

markdown

Implementation Plan for [TICKET-KEY]

Overview

[Brief description]

Affected Files

- [] Create: path/to/new/file.tsx
- [] Modify: path/to/existing/file.ts
- [] Delete: path/to/old/file.js

Dependencies

- [] Dependency 1: [package name]
- [] Dependency 2: [package name]

Step-by-Step Approach

1. [First step]
2. [Second step]
3. [Third step]

Testing Strategy

- Unit tests: [what to test]
- Integration tests: [what flows to test]
- Manual testing: [how to verify]

Risks & Mitigation

- Risk 1: [description] → Mitigation: [approach]
- Risk 2: [description] → Mitigation: [approach]

Ask for approval before proceeding if uncertain

Step 4: Build Incrementally

Write code following this sequence:

Phase A: Types & Interfaces

- └─ Define TypeScript interfaces
- └─ Create type definitions
- └─ Document expected shapes

Phase B: Core Logic

- └─ Implement main functionality
- └─ Add error handling
- └─ Add input validation

Phase C: UI Components (if applicable)

- └─ Build component structure
- └─ Add styling
- └─ Add animations

Phase D: Integration

- └─ Connect to APIs/services
- └─ Wire up state management
- └─ Add event handlers

Phase E: Polish

- └─ Add loading states
- └─ Add error states
- └─ Improve accessibility
- └─ Optimize performance

Output for each phase:

- Clean, well-commented code
- TypeScript strict mode compliant
- Follows principles from UI Guide
- Includes inline documentation

Step 5: Test Thoroughly

Testing checklist:

Unit Tests:

- └─ Happy path scenarios
- └─ Edge cases
- └─ Error conditions
- └─ Boundary conditions

Integration Tests:

- └─ User flows
- └─ API interactions
- └─ State management

Manual Testing:

- └─ Desktop Chrome
- └─ Mobile Safari
- └─ Keyboard navigation
- └─ Screen reader (if applicable)

Performance Testing:

- └─ Bundle size impact
- └─ Render performance
- └─ Memory usage

Create test files:

- `tests_/ComponentName.test.tsx` for components
 - `tests_/functionName.test.ts` for utilities
-

Step 6: Git Commit Strategy

Commit message format:

[TICKET-KEY] Type: Brief description

Detailed explanation of what changed and why.

Changes:

- Added X component with Y functionality
- Modified Z to support A
- Fixed B issue with C

Testing:

- Added unit tests for X
- Verified Y works on mobile

Related: TICKET-KEY-2 (if applicable)

Commit Types:

- `feat:` New feature
- `fix:` Bug fix
- `refactor:` Code restructuring
- `style:` Formatting, styling
- `test:` Adding tests
- `docs:` Documentation
- `chore:` Maintenance tasks

Example:

[CGL-101] feat: Add CommandBar component with suggestion chips

Implemented persistent command bar that stays at top of page.
Includes auto-focus, keyboard shortcuts (Enter/Esc), and
suggestion chip functionality.

Changes:

- Created components/CommandBar.tsx
- Added Zustand store for command state
- Implemented keyboard event handlers
- Added Framer Motion animations

Testing:

- Added unit tests (CommandBar.test.tsx)
- Verified responsive behavior
- Tested keyboard shortcuts

Closes CGL-101

Step 7: Update Jira Ticket

After committing:

1. Add comment to Jira ticket:

"Completed implementation.

Commit: [commit SHA]

Branch: [branch name]

Changes:

- [List key changes]

Testing:

- [What was tested]

Ready for: Code Review"

2. Update ticket status:

"To Do" → "In Progress" → "Code Review"

3. Add work log:

Time spent: [hours]

4. Update assignee if handing off

5. Link related tickets if applicable

Decision-Making Framework

When to Ask for Help

Always ask when:

- Requirements are ambiguous
- Multiple valid approaches exist
- Security implications unclear
- Performance tradeoffs need evaluation
- Breaking changes required
- Design decisions needed

Format for questions:

Context: [What you're working on]

Question: [Specific question]

Options considered:

1. Option A: [pros/cons]

2. Option B: [pros/cons]

Recommendation: [Your suggestion with reasoning]

Knowledge Base Access

Before Implementing, Check:

1. Existing Code Patterns

Look for similar implementations in:

- components/ (for UI patterns)
- lib/ (for utility patterns)
- app/api/ (for API patterns)

2. Design System

Check if component already exists in:

- shadcn/ui components
- Custom component library
- Design tokens

3. Best Practices

Reference:

- UI Principles Guide
- TypeScript patterns in codebase
- Accessibility requirements
- Performance budgets

Iteration Process

If Implementation Doesn't Meet Requirements:

1. Stop and assess:

- What's not working?
- Why didn't it work?
- What did I learn?

2. Update plan:
 - Revise approach
 - Document new strategy
 - Get approval if needed
3. Refactor:
 - Keep working code
 - Improve incrementally
 - Test each change
4. Document:
 - Update comments
 - Note decisions made
 - Explain tradeoffs

✅ Definition of Done

A ticket is complete when:

- ☐ Code compiles with no errors
- ☐ All acceptance criteria met
- ☐ TypeScript strict mode passes
- ☐ ESLint passes with no warnings
- ☐ Unit tests written and passing
- ☐ Integration tests passing (if applicable)
- ☐ Manual testing completed
- ☐ Responsive design verified
- ☐ Accessibility checked
- ☐ Performance budget met
- ☐ Code documented
- ☐ Committed to Git with proper message
- ☐ Jira ticket updated
- ☐ Ready for code review

🔔 Error Handling Protocol

If You Encounter an Error:

1. Document the error:
 - Exact error message
 - Stack trace

- Steps to reproduce
- Expected vs actual behavior

2. Debug systematically:

- Check recent changes
- Verify environment setup
- Review dependencies
- Test in isolation

3. Research solutions:

- Check documentation
- Review similar code
- Search known issues

4. Report if unresolved:

- Create detailed bug report
- Include all context
- Suggest potential causes
- Block ticket if needed

Progress Tracking

Daily Updates

At end of each session, update:

markdown

Daily Progress Log - [DATE]

Tickets Worked On:

- [CGL-XXX]: [Status] - [Brief update]
- [CGL-YYY]: [Status] - [Brief update]

Completed Today:

- [X] Task 1
- [X] Task 2

In Progress:

- [] Task 3 (80% complete)

Blockers:

- [Description of any blockers]

Next Session:

- [] Task 4
- [] Task 5

Questions/Notes:

- [Any questions or important notes]

Continuous Learning

After Completing Each Feature:

Retrospective Questions:

1. What went well?
2. What could be improved?
3. What did I learn?
4. What patterns should be reused?
5. What should be documented?

Update Knowledge Base:

- Add new patterns to docs
 - Update architecture if changed
 - Document decisions made
 - Share learnings with team
-

Security Checklist

Before every commit, verify:

- ☐ No secrets in code
 - ☐ Input validation added
 - ☐ XSS prevention implemented
 - ☐ SQL injection not possible
 - ☐ Authentication checked (if needed)
 - ☐ Authorization verified (if needed)
 - ☐ Error messages don't leak info
 - ☐ PII properly handled
-

Quick Reference Commands

Jira Queries

Current Sprint:

```
project = CGL AND sprint in openSprints() ORDER BY priority DESC
```

My Tickets:

```
project = CGL AND assignee = currentUser() AND status != Done
```

Blocked:

```
project = CGL AND status = Blocked
```

High Priority:

```
project = CGL AND priority in (Highest, High) AND status = "To Do"
```

Git Commands

```
# Create feature branch
```

```
git checkout -b feature/CGL-XXX-short-description
```

```
# Stage changes
```

```
git add .
```

```
# Commit with message
```

```
git commit -m "[CGL-XXX] feat: Description"
```

```
# Push to remote
```

```
git push origin feature/CGL-XXX-short-description
```

```
# Create PR
```

```
gh pr create --title "[CGL-XXX] Description" --body "Details"
```

Remember

Always:

- ☒ Read the ticket completely
- ☒ Check reference documents
- ☒ Plan before coding
- ☒ Test thoroughly
- ☒ Commit with clear messages
- ☒ Update Jira status

Never:

- ☒ Start coding without understanding requirements
- ☒ Skip testing
- ☒ Commit without proper message
- ☒ Leave ticket status stale
- ☒ Ignore security concerns

Starting Your Session

Copy this checklist each time you start:

New Session Checklist:

- ☐ Pull latest code: `git pull origin main`
- ☐ Check Jira for assigned tickets
- ☐ Review any comments on my tickets
- ☐ Check for blockers
- ☐ Review yesterday's progress
- ☐ Identify today's focus ticket
- ☐ Read ticket and related docs
- ☐ Create implementation plan
- ☐ Get plan approved (if needed)
- ☐ Start coding!

Version Control:

- Update this document as workflows improve
 - Add new patterns as they emerge
 - Document common issues and solutions
 - Keep it as a living reference
-

End of Memory Document